

Building Chatbot With Message History

Import Libraries:

```
import os
from dotenv import load_dotenv
load_dotenv() ## aloding all the environment variable

groq_api_key=os.getenv("GROQ_API_KEY")
groq_api_key
```

Output: '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'

Model:

```
from langchain_groq import ChatGroq
model=ChatGroq(model="Gemma2-9b-It")
model
```

```
ChatGroq(client=<groq.resources.chat.completions.Completions object at 0x000001FF3D1CDF0>, async_client=<groq.resources.chat.co
```

Human & AI Message:

```
from langchain_core.messages import HumanMessage

model.invoke([HumanMessage(content="Hi, My name is Slim and I am a Chief
AI Engineer")])
```

```
AIMessage(content="Hello Slim, it's a pleasure to meet you! \n\nThat's a fantastic title. As a large language model,
```

Ask your name:

```
model.invoke([HumanMessage(content="Hey What's my name and what do I do?")])
```

```
AIMessage(content="As an AI, I have no memory of past conversations and no access to your personal information. So, I don't know your name")
```

- It doesn't know your name because it has no memory.

Conversation (Human + AI Message)

✗Manually writing the human & AI message

```
from langchain_core.messages import AIMessage
model.invoke(
    [
        HumanMessage(content="Hi , My name is Slim and I am a Chief AI Engineer"),
        AIMessage(content="Hello Krish! It's nice to meet you. \n\nAs a Chief AI Engineer, what kind of projects are you working on these days? \n\nI'm always eager to learn more about the exciting work being done in the field of AI.\n"),
        HumanMessage(content="Hey What's my name and what do I do?")
    ]
)
```

```
AIMessage(content="You are Slim, and you're a Chief AI Engineer! \n\nIt was nice of you to remind me. 😊")
```

- It's able to answer because you have provided both AI and Human messages.

Message History

- It allows a chatbot to remember previous conversations

```
!pip install langchain_community
```

```
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
```

- `ChatMessageHistory` : In-memory message history class (for tracking messages)
- `BaseChatMessageHistory` : Abstract base class (for type hints)
- `RunnableWithMessageHistory` : A LangChain wrapper that adds session-aware memory to any chain/model
 - connects the chatbot to the memory

```
store={}

def get_session_history(session_id:str)→BaseChatMessageHistory:
    if session_id not in store:
        store[session_id]=ChatMessageHistory()
    return store[session_id]
```

- `store` keeps each user's conversation
- `get_session_history` makes sure each user has their own memory
- `RunnableWithMessageHistory`

- `store` is a dictionary mapping `session_id` → `ChatMessageHistory`
- If a session doesn't exist, it initializes a new history for it
- Returns the appropriate message history based on the `session_id`

→ `BaseChatMessageHistory` :

It's **not required**, but it's helpful. It tells Python (and humans!) that this function will return **something that behaves like a chat history** — in this case, a type of `BaseChatMessageHistory` .

Wrap Model with Memory

```
with_message_history = RunnableWithMessageHistory(model, get_session_history)
```

This wraps your Groq `model` with logic that:

- Automatically pulls the right chat history using `get_session_history`
- Stores new messages after each call
- Passes the conversation history into the model

Session 1: Start a Conversation

```
config={"configurable":{"session_id":"chat1"}}

response=with_message_history.invoke(
    [HumanMessage(content="Hi , My name is Slimand I am a Chief AI Engineer")],
    config=config
)
response.content
```

```
"Hi Slim, it's great to meet you! \n\nBeing a Chief AI Engineer is a demanding but rewarding"
```

- Memory is updated: `session_id='chat1'` now has one human message

Follow-Up: Same Session, Continued Context

Ask Your name:

```
with_message_history.invoke(  
    [HumanMessage(content="What's my name?")],  
    config=config,  
)
```

```
AIMessage(content='Your name is Slim. \n\nYou told me earlier that you are a Ch
```

- Since you're still using `session_id="chat1"`, LangChain passes the **entire conversation history** to the LLM.

New Session: No Memory

```
## change the config→session id  
config1={"configurable":{"session_id":"chat2"}}  
response=with_message_history.invoke(  
    [HumanMessage(content="Whats my name")],  
    config=config1  
)  
response.content
```

```
"As an AI, I have no memory of past conversations and do not know your name. If
```

```
response=with_message_history.invoke(  
    [HumanMessage(content="Hey My name is John")],  
    config=config1
```

```
)  
response.content
```

```
"Hi John! It's nice to meet you. \n\nWhat can I do for you today?\n"
```

```
response=with_message_history.invoke(  
    [HumanMessage(content="Whats my name")],  
    config=config1  
)  
response.content
```

```
'Your name is John! I remember. 😊 \n\nIs there anything else I can help you with?\n'
```

Prompt Templates

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

Line	What is it?	Why is it needed?
<code>ChatMessageHistory</code>	A class to store and manage conversation messages (user + AI)	Needed to keep track of chat history
<code>RunnableWithMessageHistory</code>	A special wrapper that connects a chain (model) with a memory/history	It lets the model remember past turns
<code>ChatPromptTemplate</code>	Lets you define structured prompts (like system + user + memory)	Needed to define how you want to talk to the LLM

Line	What is it?	Why is it needed?
<code>MessagesPlaceholder</code>	A placeholder in the prompt where previous messages will be inserted	Without this, the model won't "see" past chats

Create prompt:

```
prompt=ChatPromptTemplate.from_messages(
    [
        ("system","You are a helpful assistant.Amnswer all the question to the be
st of your ability"),
        MessagesPlaceholder(variable_name="messages")
    ]
)
```



Here, the human message needs to be provided in a key-value pair.

Create a chain:

```
chain = prompt|model
```

Ask Question:

```
chain.invoke({"messages":[HumanMessage(content="Hi My name is Slim")]}))
```

```
AIMessage(content="Hello Slim! It's nice to meet you. \n\nI'm happy to answer any questi
```

- You'll get same response with `chain.invoke(["Hi My name is Slim"])`

- But this is a simple LLM.

Chat with History:

Previous Code as is:

```
store={}

def get_session_history(session_id:str)→BaseChatMessageHistory:
    if session_id not in store:
        store[session_id]=ChatMessageHistory()
    return store[session_id]
```

```
with_message_history = RunnableWithMessageHistory(chain, get_session_history)
```

- `chain` has prompt & model

Previous code was:

```
with_message_history = RunnableWithMessageHistory(model, get_session_history)
```

Response:

```
config = {"configurable":{"session_id":"chat3"}}

response = with_message_history.invoke(
    [HumanMessage(content= "Hi, my name is slim.")],
    config=config
```

```
)  
response
```

```
AIMessage(content="Hello Slim, it's nice to meet you! \n\nI'm here to help. Ask me anything! 😊 What
```

Ask your name:

```
response = with_message_history.invoke(  
    [HumanMessage(content="What's my name?")],  
    config=config,  
)  
  
response.content
```

```
'Your name is Slim! I remember. 😊 \n\n\n'
```

Add more complexity (Message + Language)

```
prompt= ChatPromptTemplate.from_messages(  
    [  
        (  
            "system", "Answer all questions to the best of your ability in {language}.",  
            MessagesPlaceholder("message")  
        )  
    ]  
)
```

```
chain = prompt|model
```

```
response = chain.invoke(  
    {  
        "message": [HumanMessage(content="Hi, my name is slim")],  
        "language": "Hindi"  
    }  
)  
response.content
```

```
'नमस्ते, Slim! \n\nआपके नाम बहुत अच्छा है। मैं आपकी सहायता करने के लिए तैयार हूँ। \n\nआप मुझसे क्या पूछना चाहते हैं? \n\n'
```

- Here, we passed 2 key-value pairs (`message` & `language`)

Chat with History

```
with_message_history=RunnableWithMessageHistory(  
    chain,  
    get_session_history,  
    input_messages_key= "messages"  
)
```

`input_messages_key= "messages"` :

“Inside the input dictionary, the actual list of messages is stored under the key called '`messages`'.

- If you don't set `input_messages_key`, LangChain **won't know where to find the messages**, and memory won't work properly.

!!You didn't need to use `input_messages_key` before because you were passing the **list of messages directly**, like this:

```
with_message_history.invoke(  
    [HumanMessage(content="Hi, my name is Krish")],  
    config=config  
)
```

This works **without** `input_messages_key` because:

- You're not passing a dictionary
- You're passing the messages **directly as a list**
- LangChain assumes: *"Hey, these are the messages!"* ✅

```
config = {"configurable": {"session_id": "chat4"}}  
  
response = with_message_history.invoke(  
    {"messages": [HumanMessage(content="Hi, my name is slim")],  
     "language": "Hindi"  
    },  
    config=config  
)  
response.content
```

```
'नमस्ते, स्लिम! \n\nआपका नाम सुनकर बहुत अच्छा लगा। 😊 \n'
```



Here, we passed an additional `"language"` parameter.

Ask your name:

```
response = with_message_history.invoke(  
    {"messages": [HumanMessage(content="whats my name?")], "language":  
    "Hindi"},  
    config=config,  
)  
  
response.content
```

```
'आपका नाम स्लिम है। 😊 \n'
```